

Primitive tool kit + orchestrator technique

Composition, primitives, and the difference between vendor walls and physical law

A doctrine by Jesse — the principle underneath the RF work, the codecs, the cameras, the GPU kit, and every platform I build. This revision is grounded in actually building it (the Sparse Context Governor), not just describing it — so the abstract claims below have working proof, and a few of them got sharper once they had to compile.

The thing in one sentence

One benign exposed function can be transformed into any function in the span, because everything breaks down to math, math composes, and a small set of primitives reaches a space vastly larger than the primitives suggest.

Everything below unpacks that sentence — and, just as important, the places where it stops being true.

1. The one move

There is a single move under all of it:

> Decompose the thing into mathematical primitives. Manipulate it in a basis where the operation is cheap and clean. Recompose.

Don't operate on the raw representation. Transform it into a domain where the structure you care about becomes separable, do the work **there**, and reconstruct from the result. This is the adapter pattern, generalized past adapters.

It's the same move whether the object is an RF signal, a codec, a weight delta, an expensive GPU function, a vendor API, or a model's context window. Take it into the transform domain, manipulate the coefficients where the operation is well-conditioned, synthesize back. An LLM, a LoRA update, an RF waveform, a camera endpoint, and a retrieval corpus are the same **kind** of object: something you factor into primitives and rebuild, rather than something you poke at in its native form.

It is not five tricks. It is one move, pointed at five sealed executors.

****The avatar adapter is the cleanest instance.**** My codec adapter sits between the camera sensor and the codec. It captures the image, converts the pixels to pure math, uses math

primitives from the kit to *replace the original math with new math*, rebuilds the math back into pixels, and hands the pixels to the codec. That's how I make cheap avatars with no heavy hardware — it runs on the CPU. The codec never knows the pixels were synthesized; it only ever sees its input language. Same move, pointed at retrieval, builds a context pack the model reads as if it were the corpus.

2. Why it generalizes: functional completeness — and the atoms underneath

The move works because of **functional completeness** — universality. A small primitive set, composed, spans an enormous space:

- NAND alone → all of boolean logic.
- A universal gate set → every quantum operation.
- Sinusoids → every signal.
- A handful of API calls → behaviors no vendor ever shipped.

The cleanest proof is in my own domain. I synthesize RF waveforms **that do not exist in nature**. No natural source emits them. But they're reachable — because they live *inside the span of the basis*. Nature not producing a thing has nothing to do with whether it's reachable. The span decides that.

The trap, and the thing the build taught me to be ruthless about: **what you call a "primitive" is usually still a function — a composite — with engines hiding inside it**. "Vectorize," "embed," "rerank," "score" all *feel* primitive. They are not. One function contains many primitives. You have to keep decomposing until what's left is genuinely atomic and agnostic.

Decomposed to the root, an entire retrieval engine is this atom set:

> **scan · hash · fold · project · scale · compare · combine · order**

And every "capability" is just a *wiring* of those atoms:

- tf-vector = scan → hash → fold(count)
- BM25 = tf-vector + fold(count across docs) + combine
- minhash = scan → hash×k → fold(min)
- **embed** = tf-vector → project → scale
- similarity = compare
- centrality = fold(adjacency) → project (iterated) → scale
- rerank = order over combine

That's functional completeness made literal: embed and BM25 and hash and rerank are not different engines — they are *the same atoms wired differently*. I have this running. The

embedder in the kit is built from the same atoms as the keyword scorer, no neural model anywhere, and it still puts related text closer than unrelated. That's the whole doctrine in one passing test: a lexicon transformed into an embedder by re-wiring atoms, not by bolting on an engine.

This is also why adding a capability is fast: you're not building an engine, you're writing a wiring.

3. The primitive is agnostic — cost lives in the generator, never the atom

Here is the correction the build forced, and it matters more than it looks.

A primitive, at its most primitive form, is agnostic and simple. If you think a primitive is going to cost you, you are thinking about it wrong — you've left an engine inside it.

"Embed" is the example that fooled me first. Embedding is not "call a 600 MB transformer." At its root it is two atoms: **vectorize** (put the thing in a space) and **compare** (measure nearness). It does not care *how* the coordinates got assigned. The neural net is just *one* way to fill the vector — a heavy, optional **generator** you plug into the ``project`` slot. It is an engine, not the primitive.

So cost is not a property of the capability. It is a property of the *generator* you back a slot with. The cheap generators are real embeddings too — feature hashing, a random projection of the term vector, MinHash, the TF vector itself — all pure math, instant, no model. Swap a neural generator into the same slot and you've bought fidelity at the price of weight and flakiness. Same primitive, different generator, wildly different cost.

The practical payoff: you *always* do the cheap thing (it's free), and you *escalate* to the heavy generator only for the residual the cheap one can't separate. "Selective embedding" isn't a special feature you bolt on — it falls out for free the moment embed is a primitive with a swappable generator instead of a monolithic engine.

4. The primitive itself is transformable — reinforce, and fuse

The atoms aren't sacred either. An atom is still math, so it too can be transformed — *within the laws of physics.*

Reinforce it. My LoRA primitive, trained into a LoRA, increases signal in a degraded model. I fortify RF signals by reinforcing the primitives themselves. The retrieval version: take the ``project`` atom from a *random* projection to a *corpus-learned* one — IDF-whitening first (the corpus's own statistics flatten the common-term background before you match — the

matched-filter move), then latent factors (LSA over `project`). Same atom, reinforced with structure learned from the data it operates on.

****Fuse a second capability into it.**** I can build a CB RF transport lane *into* a LoRa signal — not by adding a waveform on top, but by rebuilding the signal's math to include the CB transport. The capability lives inside the primitive's math now, not beside it.

But fusion and reinforcement are never free, and the bill comes due in a very specific way — which is the subject of the next section.

5. Two kinds of walls — and the conserved cost that tells them apart

Here's the whole craft: telling these apart. Most walls are painted on. A few are law.

****Painted-on walls**** are *existence*-limited, not composition-limited. The vendor declines to expose a call, or changes the locks: a thin API, a capability toggle switched off, an auth scheme that rotates, signed firmware, an endpoint that won't hand you the score it computes internally. The function isn't thin — it's *absent*. The response is to ****route around****: re-implement the call, host-stage the transfer the disabled feature won't do, drive the engine yourself so there's nothing to intercept. The fence is mostly paint.

****Real walls**** are law — information theory, computability, and physics — and composition genuinely cannot reach past them. My own domain has the cleanest example: I can synthesize signals that don't exist in nature, but I cannot synthesize one arbitrarily sharp in both time *and* frequency. The uncertainty principle forbids it. Shannon capacity, the entropy floor, uncomputability, NP-hardness — same character. You don't route around these. You operate *inside* them, which is what good RF work does, respectfully, every day.

****The tell — and this is the sharp part — is conservation.**** When you hit a real wall, the cost of what you wanted comes back as a *conserved quantity*. I can fuse the CB lane into the LoRa signal, and it works — but it *shortens the signal's reach*, because energy and bandwidth are conserved. I didn't lose range to bad engineering. I *spent* it. That's the signature of a real wall: capability is not denied, it's *paid for* in a currency that doesn't refund.

Every domain has its conserved currency, and naming it keeps you honest:

- ****RF:**** range, paid in energy/bandwidth (time–bandwidth).
- ****Compression:**** distortion, paid against bits (rate–distortion).
- ****Retrieval / a fortified primitive:**** *generality*. A corpus-tuned projection is stronger *here* and weaker *everywhere else*. That "shortened reach" is the LoRa range bill, paid in information instead of RF energy. You don't get the corpus's semantics for free — you spend the model's breadth to buy them.

When the cost is a conserved quantity, it's a real wall and you stop pushing. When there's no conserved bill — just a missing call, a flipped toggle, a withheld score — it's paint, and you route around it. ****Telling which is which is the entire job.**** A thin API is an incomplete basis; CUDA is a near-complete one; physical law is the dimension the space simply doesn't contain.

6. Harvest primitives from any domain — the boundary is paint too

The atoms don't know what field they came from. So the kit should never be sourced from one discipline's playbook — it should be sourced from **anywhere the same atom shows up better-developed.**

> One naturally assumes a different domain is a different domain, with no relevance. You'd be surprised how wrong that assumption is.

"That's a different field, it won't apply" is the same illusion as "that's a walled garden" — it's paint. The math is the truth; the domain label is just where someone happened to discover the atom first. The harvests that upgraded the governor came straight out of unrelated fields:

- ****Graphics → mipmaps / level-of-detail:**** a coarse summary tier you read for breadth, sampling fine only where the query looks closely. The coarse-to-fine context pack is a solved graphics primitive, not a new invention.
- ****Physics → diffusion / spreading activation:**** activation that **decays** with distance from the query, so expansion falls off instead of flooding the whole corpus. (This replaced a hard k-hop walk that lit up nearly everything on real data.) Percolation theory even hands you the threshold where a flood tips into "everything."
- ****RF → matched filter + whitening:**** retrieval **is** a matched filter (correlate the query template against documents). The filter is only optimal after you whiten the background — which is what IDF crudely approximates, and what a proper whitening does sharply.
- ****Information theory → PMI:**** the corpus's own notion of synonyms, learned from co-occurrence, model-free.
- ****Databases / streaming → count-min, HyperLogLog, Bloom:**** corpus statistics in tiny fixed space, so the cheap kit scales without ever touching a model.

The same honest filter applies as with walls: harvest the primitive that fixes a **measured** gap; hold the rest as inventory until something needs it. A clever transfer that fixes nothing is just a cute analogy.

7. How it cashes out: dumb primitives, a brain that routes, one recursive contract

The doctrine becomes the primitive-kit-plus-orchestrator shape I use everywhere.

****Primitives are mechanism with no decisions.**** Each does one thing and speaks the engine's vocabulary fluently — the zero-copy buffer handle, the explicit cross-device transfer, the stream-ordered pool, the `scan`, the `project`, the `compare`. Dumb on purpose. And cheap on purpose — if one strains, it isn't a primitive yet.

****The orchestrator is all policy and no mechanism**** — inventory, placement, where to cut the graph, how to route each edge, when to spill, what to prefetch, **which generator to pay for and only when the cheap kit can't reach.**

Two rules keep it clean:

1. ****Keep the primitives dumb.**** A hybrid is a hybrid **system** — dumb primitives plus a brain that routes between them — never a hybrid **mechanism**, one clever object trying to be two things at once. This is exactly how you build a ****hybrid lexicon****: the lexical substrate stays simple, and the orchestrator hands it a tool — hash, rerank, embed — the moment it needs to do something it can't do natively. The lexicon doesn't **route to** another engine; it **becomes** that capability by picking up the atom. With the right kit, the lexicon can hash, rerank, and embed.

2. ****Keep cost explicit.**** Transparent migration (CUDA Unified Memory) is the wrong convenience: a page fault can fire a silent transfer mid-computation — the exact unpredictable stall the design exists to remove. I hit the identical bug in retrieval: a silent always-embed choked a small local model under load and starved everything downstream. The fix is the same in both domains — make the cost an **explicit, rationed decision** the orchestrator makes, never a hidden default. Explicit cost beats invisible cost, every time.

And the contract is ****recursive.**** A primitive and an orchestrator have the same shape one level up: the orchestrator is itself a tool a higher orchestrator composes. So SCG is an engine you drive from outside; the vector store is a tool; the bus is a tool; the model is a sealed codec fed tokens. Nothing is fixed furniture. There is nothing in the stack you can't decompose and rebuild into what you actually need — bounded only by the conserved-cost walls of Section 5.

The shape of the whole thing

Devs spend their lives trying to crack the walled garden. The trap is that cracking **feels** like the more powerful move — it promises unconditional control — while composing from outside looks like settling for the engine's vocabulary. The instinct runs backwards. The thing that actually ships looks like the compromise.

But everything you pry open, you own only until the next release — the driver patch breaks, the assembler dies each new architecture, the shim shatters when an ABI shifts. You're renting

control from a vendor who reserves the right to change the locks. Compose from outside and the vocabulary is a stable contract: they can rewrite every internal and your composition still runs, because you only ever depended on the input language. You traded the *illusion* of total control for the *reality* of a durable interface. That's not the lesser deal. That's the entire deal.

It only works when the vocabulary is expressive enough — and the craft is knowing which gardens have a front door wide enough to walk through. CUDA does. Blink did. Vello does. The fence is mostly painted on. The few real bars are mathematical and physical law, and they announce themselves by charging you in a conserved quantity. Telling which is which is the whole job — and it's exactly the instinct a life in RF builds, because RF is where you live pressed right up against the real ones.